

Challenge:

We intercepted a communication between Cipher and some 3 associates: Rivest, Shamir and Adleman. We were only able to retrieve a file.

ORDER: Get the secret key from the recovered file.

```
from Crypto.Util.number import *
from flag import FLAG

def primo(n):
    n += 2 if n & 1 else 1
    while not isPrime(n):
        n += 2
    return n

p = getPrime(1024)
q = primo(p)
n = p * q
e = 0x10001
d = inverse(e, (p-1) * (q-1))
c = pow(bytes_to_long(FLAG.encode()), e, n)
#c =
359111666431198697688229938559813544743524646070650088724176955508841635968
278784453241494357379499369997603550488466283495684684986319964310425442388
604048930717724020087744332503646902073773473525200989020386070356546702749
490617845525748756090259982336457107262767327466346016725899444499973216416
341306970560391891291802934190673124961839056063129451646007206028209633818
836321801831055825633350207548113259347478427252931814198301668476261185335
005813542017743651164659370354199490463240589167584898735544449033816263636
080643786267932161213614743757879969663063193327776726353052635453289865593
7702383789647510
#n =
159562501620631698192829474437432743700486432744167426553488178239733838293
647005739547092563912458265131077847139303789635516477067774797782854733026
656644464060614856168841959246315821306331375749532933679279912836695628959
566998071569580715408180231223621630662532409251218010137676600747480212387
903914544297108044974327838526015493995230029680049895377172834408683126480
426761037450614317999271201535232603282859534251366757941926044068658787952
093269987671749186425997097286174527054921222438535481099143991853698132898
273422940842039336156453907288906981534903186365444747147007965697464882094
38597446475170891
```

First, we inspect the code to see what information it can tell us about the encryption

```
def primo(n):
    n += 2 if n & 1 else 1 # get the next odd number after n
    while not isPrime(n):
        n += 2 # iterates through the odd numbers until it finds a prime
    return n
```

The primo() function gets the next consecutive prime number after the one entered into the function.

```
p = getPrime(1024) # assign p a random 1024 bit prime number
q = primo(p) # assign q the next consecutive prime number after p
n = p * q
e = 0x10001 # binary representation of 17
d = inverse(e, (p-1) * (q-1))
```

These are the variables needed for the RSA function. If we find the values of p and q we will be able to break the encryption. Since p and q are consecutive primes, they will end up being very close to each other, meaning we can use Fermat's factorization algorithm to find their values.

```
c = pow(bytes_to_long(FLAG.encode()), e, n)
```

This section lets us know that we'll have to convert our answer from long back to bytes so that we can retrieve the original message.

```
#c =
359111666431198697688229938559813544743524646070650088724176955508841635968
278784453241494357379499369997603550488466283495684684986319964310425442388
604048930717724020087744332503646902073773473525200989020386070356546702749
490617845525748756090259982336457107262767327466346016725899444499973216416
341306970560391891291802934190673124961839056063129451646007206028209633818
836321801831055825633350207548113259347478427252931814198301668476261185335
005813542017743651164659370354199490463240589167584898735544449033816263636
080643786267932161213614743757879969663063193327776726353052635453289865593
7702383789647510

#n =
159562501620631698192829474437432743700486432744167426553488178239733838293
647005739547092563912458265131077847139303789635516477067774797782854733026
656644464060614856168841959246315821306331375749532933679279912836695628959
566998071569580715408180231223621630662532409251218010137676600747480212387
903914544297108044974327838526015493995230029680049895377172834408683126480
426761037450614317999271201535232603282859534251366757941926044068658787952
093269987671749186425997097286174527054921222438535481099143991853698132898
273422940842039336156453907288906981534903186365444747147007965697464882094
38597446475170891
```

The final 2 variables we need, with c being the encrypted message, and n the public key (alongside e, given before).

Now we create our own function:

```
# Implementation of Fermat's factorization

from math import ceil, isqrt

#This function finds the value of a and b
#and returns a+b and a-b
def FermatFactors(n):

    # since fermat's factorization applicable
    # for odd positive integers only
    if(n<= 0):
        return [n]

    # check if n is a even number
    if(n & 1) == 0:
        return [n / 2, 2]
```

```

a = ceil(isqrt(n))

#if n is a perfect root,
#then both its square roots are its factors
if(a * a == n):
    return [a, a]

while(True):
    b1 = a * a - n
    b = isqrt(abs(b1))
    if(b * b == b1):
        break
    else:
        a += 1
return [a-b, a + b]

#run the function
n=1595625016206316981928294744374327437004864327441674265534881782397338382
936470057395470925639124582651310778471393037896355164770677747977828547330
266566444640606148561688419592463158213063313757495329336792799128366956289
595669980715695807154081802312236216306625324092512180101376766007474802123
879039145442971080449743278385260154939952300296800498953771728344086831264
804267610374506143179992712015352326032828595342513667579419260440686587879
520932699876717491864259970972861745270549212224385354810991439918536981328
982734229408420393361564539072889069815349031863654447471470079656974648820
9438597446475170891

print(FermatFactors(n))

```

We apply Fermat's factorization to the value of n given to us, running this function will give us the values of p and q:

```

[12631805160808636308643616767034426339408047082059561443160134032277084207
728156127043054645818192704703510717149544373305944619732121303911405887907
411643500427574667789518416641607243942585143668523774937610542861375281676
047990627066260984542034795514687057655389017129764652333875741077290537271
1647921869,
126318051608086363086436167670344263394080470820595614431601340322770842077
281561270430546458181927047035107171495443733059446197321213039114058879074
116435004275746677895184166416072439425851436685237749376105428613752816760
479906270662609845420347955146870576553890171297646523338757410772905372711
647922039]

```

Now that we have all the relevant variables we can apply the following function to retrieve our original message: $\text{answer} = c^d \bmod n$

```
# Final calculation to get our result
from Crypto.Util.number import *
import math

p =
126318051608086363086436167670344263394080470820595614431601340322770842077
281561270430546458181927047035107171495443733059446197321213039114058879074
116435004275746677895184166416072439425851436685237749376105428613752816760
479906270662609845420347955146870576553890171297646523338757410772905372711
647921869
q =
126318051608086363086436167670344263394080470820595614431601340322770842077
281561270430546458181927047035107171495443733059446197321213039114058879074
116435004275746677895184166416072439425851436685237749376105428613752816760
479906270662609845420347955146870576553890171297646523338757410772905372711
647922039
n = p * q
given_n =
159562501620631698192829474437432743700486432744167426553488178239733838293
647005739547092563912458265131077847139303789635516477067774797782854733026
656644464060614856168841959246315821306331375749532933679279912836695628959
566998071569580715408180231223621630662532409251218010137676600747480212387
903914544297108044974327838526015493995230029680049895377172834408683126480
426761037450614317999271201535232603282859534251366757941926044068658787952
093269987671749186425997097286174527054921222438535481099143991853698132898
273422940842039336156453907288906981534903186365444747147007965697464882094
38597446475170891
c =
359111666431198697688229938559813544743524646070650088724176955508841635968
278784453241494357379499369997603550488466283495684684986319964310425442388
604048930717724020087744332503646902073773473525200989020386070356546702749
490617845525748756090259982336457107262767327466346016725899444499973216416
341306970560391891291802934190673124961839056063129451646007206028209633818
836321801831055825633350207548113259347478427252931814198301668476261185335
005813542017743651164659370354199490463240589167584898735544449033816263636
080643786267932161213614743757879969663063193327776726353052635453289865593
7702383789647510
e = 0x10001

d = inverse(e, (p-1) * (q-1))
```

```
x = pow(c, d, n)
print(n == given_n) #returns true if our values for p and q are correct
answer = long_to_bytes(x).decode() #remember to convert it back to bytes!
print(answer)
```

We use the given values for c, e, d, n, and include our calculated values for p and q. Comparing $p * q$ with the value of n to confirm that they're correct is not necessary, although it never hurts to double check!

Running this code gives us the answer: **THM{Just_s0m3_small_amount_of_RSA!}**